Accelerated Systems Principles and Practice
Coursework 2

B283069[1]

[1]University of Edinburgh

Abstract. *This report presents a CUDA implementation of the 3D wave-equation coursework application on single-node multi-GPU systems. The work focuses on correctness against the provided CPU reference and performance on A100 and H100 GPUs for problem sizes 256–2500 and device counts up to 8.*

1. Design and Process

1.1. Programming Model Choice

Chosen model: CUDA. The submission uses CUDA because it gives direct control of device memory, kernel launch configuration, and synchronization primitives needed for optimizing stencil updates and halo exchange behavior. The codebase already has a clear MPI decomposition structure, so CUDA integration at rank level is straightforward: each MPI rank owns one local subdomain and executes device work for that subdomain.

OpenMP target offloading was considered, but it is less coherent with the optimization goals of this MPI-decomposed code. In this coursework, performance is strongly affected by how device-side buffers are managed around halo exchange and by how communication/computation overlap is orchestrated. CUDA exposes these mechanisms explicitly (streams, events, device allocations, asynchronous copies), while OpenMP offloading tends to hide more of the runtime behavior behind directives. For this project, that reduced explicit control is a disadvantage because the bottlenecks are communication-path details rather than simple loop parallelism.

SYCL was also considered, but the coursework target is specific NVIDIA hardware (A100/H100) rather than portability across vendors. Since the objective is to maximize node performance on a known platform, hardware-agnostic abstractions are less valuable than direct platform-specific control. In this context, CUDA better matches the purpose of the assignment: optimize for a fixed NVIDIA environment instead of prioritizing cross-vendor portability.

1.2. Implementation Baseline

The provided starter structure is MPI-first and uses domain decomposition with halo exchange on every timestep:

- 3D process grid (‘-mpi px,py,pz‘) determines local subdomains.
- CPU and GPU implementations are executed and compared for correctness (‘Number of differences detected‘).
- Initial GPU backend was effectively CPU-style logic and did not exploit device-side communication/computation structure.

1.3. Optimization Steps and Rationale

The implementation was developed incrementally with correctness checks after each step.

Stage 1: CUDA compute kernel The first target was to move the dominant arithmetic work (7-point stencil + damping update) from CPU loops into a single CUDA kernel operating on rank-local data. The objective was to establish a correct device execution path while preserving the original timestep ordering (‘halo exchange → update → advance‘). At this stage, MPI decomposition, data layout, and host-side halo orchestration were intentionally left unchanged to reduce integration risk. This created a clean baseline for later optimization: functional CUDA execution with minimal algorithmic change.

Stage 2: Halo-only communication path After Stage 1, profiling showed that communication-related transfers dominated runtime because the whole active field was copied device↔host each timestep to perform MPI halo exchange. The main design decision in Stage 2 was to reduce transfer volume to the true communication requirement: only the six boundary faces. This was implemented with CUDA face pack/unpack kernels and pinned host buffers for host-side MPI exchange. In effect, each timestep transfers halo faces only, instead of full 3D arrays, reducing PCIe traffic and improving device utilization. Correctness remained unchanged because the numerical update formula and halo semantics were preserved.

Stage 3: Communication/computation overlap Stage 3 targeted latency hiding rather than data-volume reduction. The local domain update was split into two kernels:

- an *interior* kernel for points that do not depend on freshly received halos,
- a *boundary* kernel for points adjacent to process boundaries.

Halo exchange was executed on a communication stream while interior work executed on a compute stream. A CUDA event was used to synchronize only the boundary phase with halo completion. The rationale is that interior arithmetic can proceed concurrently with communication, so end-to-end timestep time approaches ‘max(interior compute, halo exchange) + boundary compute‘ instead of a fully serialized sum. This stage improved strong-scaling behavior by reducing visible communication overhead per timestep.

1.4. Correctness Method

Correctness was validated using the built-in CPU reference comparison:

- ‘Number of differences detected = 0‘ was required for acceptance.
- Validation was performed across multiple process layouts (‘1,1,1‘, ‘2,1,1‘, ‘2,2,1‘, ‘1,2,2‘) and multiple shape sizes.
- Restart/checkpoint paths were also tested during development.

1.5. Experimental Method

Current measurements were collected with one timing chunk (`-nsteps 100 -out_period 100`) so runtime equals a single 100-step interval. The same command format was used per case except where infrastructure constraints forced MPI transport changes.

- GPU platform currently measured: A100 80GB.
- Process layouts used: ‘1,1,1‘ (1 rank/GPU) and ‘2,2,1‘ (4 ranks/GPUs).
- IO disabled during timing (‘-io‘ default off).
- CPU+GPU comparison mode used for the runs listed in this draft section.

2. Results

2.1. A100 Results (from `A100-Benchmarks.csv`)

GPUs	Shape	CPU time (s)	CUDA time (s)	GPU/CPU speedup
1	128^3	5.157e-02	4.241e-03	12.16×
1	256^3	4.504e-01	4.223e-02	10.67×
1	512^3	3.261e+00	3.457e-01	9.43×
1	1024^3	2.127e+01	2.814e+00	7.56×
2	128^3	2.182e-02	3.434e-03	6.35×
2	256^3	2.064e-01	2.272e-02	9.08×
2	512^3	1.783e+00	2.062e-01	8.65×
2	1024^3	1.406e+01	1.591e+00	8.84×
4	128^3	8.772e-03	3.109e-03	2.82×
4	256^3	1.199e-01	2.015e-02	5.95×
4	512^3	9.719e-01	1.670e-01	5.82×
4	1024^3	8.015e+00	1.237e+00	6.48×

Table 1. A100 benchmark summary extracted from `A100-Benchmarks.csv`.

2.2. H100 Results (from `H100-Benchmarks.csv`)

GPUs	Shape	CPU time (s)	CUDA time (s)	GPU/CPU speedup
1	128^3	4.669e-02	3.244e-03	14.39×
1	256^3	8.434e-01	4.888e-02	17.25×
1	512^3	5.521e+01	2.488e+00	22.19×
2	128^3	2.047e-02	2.299e-03	8.90×
2	256^3	4.249e-01	2.370e-02	17.93×
2	512^3	3.825e+00	2.213e-01	17.28×
2	1024^3	3.234e+01	1.730e+00	18.69×
4	128^3	9.633e-03	1.836e-03	5.25×
4	256^3	1.569e-01	1.092e-02	14.37×
4	512^3	1.547e+00	8.904e-02	17.37×
4	1024^3	1.319e+01	7.629e-01	17.29×

Table 2. H100 benchmark summary extracted from `H100-Benchmarks.csv`.

2.3. Discussion of Results

- All listed runs achieved zero differences against CPU reference.
- CUDA outperforms CPU in all measured cases.
- From `A100-Benchmarks.csv`, GPU-over-CPU speedup ranges from 2.82× to 12.16×.
- From `H100-Benchmarks.csv`, GPU-over-CPU speedup ranges from 5.25× to 22.19×.
- At fixed GPU count, speedup generally increases with problem size, consistent with better amortization of communication and launch overheads as per-rank workload increases.
- A preliminary 8-GPU run required different MPI transport flags because of UCX shared-memory allocation issues. For fair scaling comparisons, 8-GPU data will be regenerated using consistent runtime configuration after infrastructure adjustments.

The tables above are the current consolidated baseline used for the performance analysis plots.

2.4. Performance Graphs

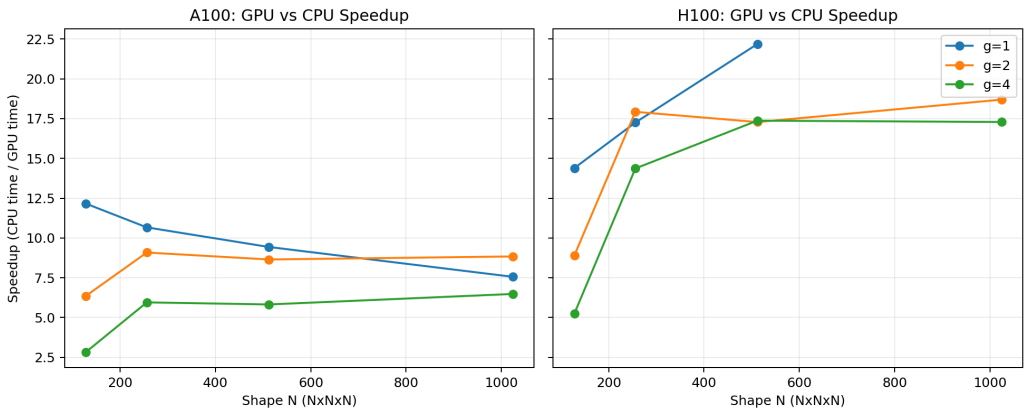

Figure 1. GPU-over-CPU speedup versus problem size for A100 and H100, grouped by GPU count.

Interpretation (Speedup vs CPU). Both platforms show clear acceleration over CPU across all measured cases. Speedup generally increases with problem size because kernel launch and communication overheads become a smaller fraction of total timestep cost as arithmetic intensity rises. H100 tends to show stronger speedup at larger cases due to higher memory bandwidth and higher aggregate device throughput. Lower speedups at small shapes, especially at higher GPU counts, are expected: communication and synchronization overheads are proportionally larger when per-rank workloads are small.

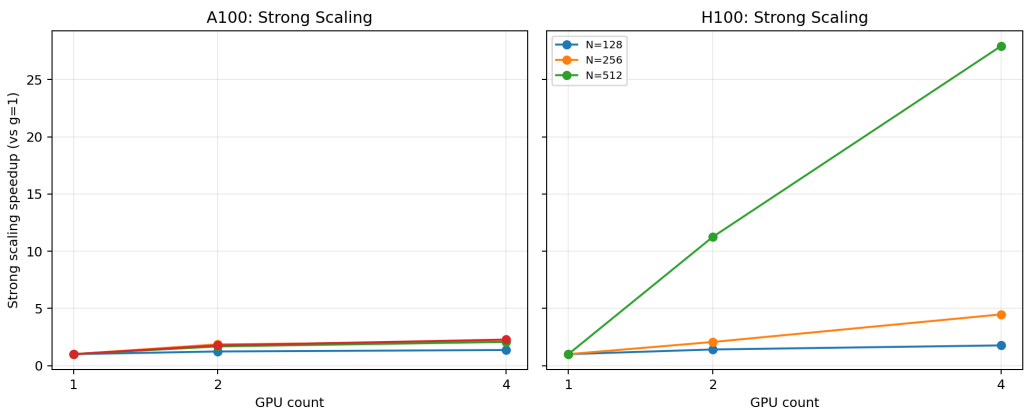

Figure 2. Strong-scaling speedup (relative to $g = 1$) for $g = 1, 2, 4$ on A100 and H100.

Interpretation (Strong Scaling). Strong-scaling speedup improves as shape increases, because larger fixed problems provide enough local work per rank to amortize halo exchange and MPI synchronization costs. For smaller shapes, scaling is sub-linear as expected for stencil applications with nearest-neighbor communication. In a finite-volume stencil update, communication volume scales with surface area while useful computation scales with volume; therefore, decomposition efficiency improves with larger local domains and degrades when each rank owns too small a block.

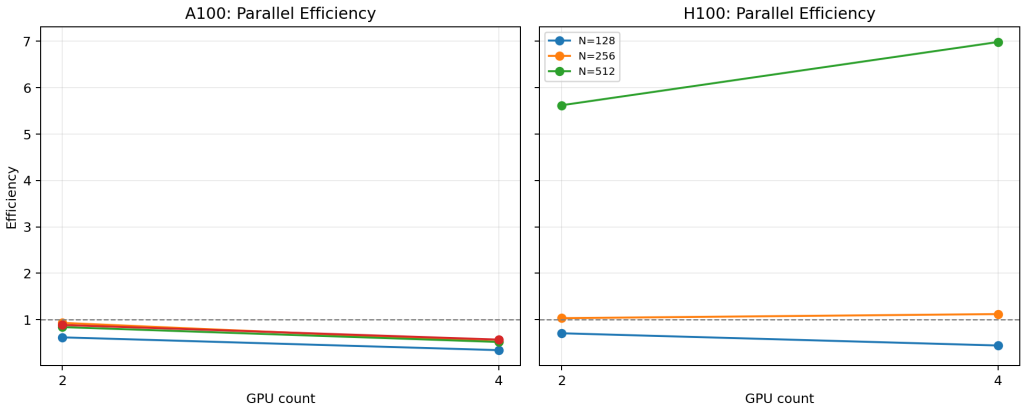

Figure 3. Parallel efficiency for $g = 2$ and $g = 4$ derived from strong-scaling runs.

Interpretation (Parallel Efficiency). Efficiency decreases when increasing GPU count at fixed shape, consistent with communication and synchronization overhead growth relative to per-rank compute. Efficiency is higher for larger shapes because each rank has more interior points, improving the compute/communication balance. The remaining spread between curves likely reflects runtime-system effects (scheduler placement, MPI transport behavior, and node-level contention), which is typical in shared production environments. Final conclusions should use repeat runs and consistent runtime flags, especially once 8-GPU measurements are completed.

3. Further Work

Estimate multi-node scaling behavior and changes needed for efficient execution across hundreds of nodes / thousands of GPUs. Address:

- Inter-node communication costs and expected scaling limits.
- Overlap and message aggregation strategies.
- Potential use of CUDA-aware MPI or GPUDirect paths.
- Domain decomposition and load-balance considerations at scale.

References

[1] NVIDIA CUDA Documentation, `https://docs.nvidia.com/cuda/`.